



nextwork.org

AI Finance Agent with Amazon Bedrock



yakubu abdullahi yusuf



Introducing Today's Project!

In this project, I will build an AI agent using Amazon Bedrock Agents that reads your spending data, categorizes expenses, calculates budgets, and generates visual charts, I'm doing this project to learn how to build autonomous AI agents using Amazon Bedrock Agents, integrate data analysis workflows into AI applications, process and categorize financial data automatically, and generate actionable insights and visualizations from real-world datasets.

Key services and concepts

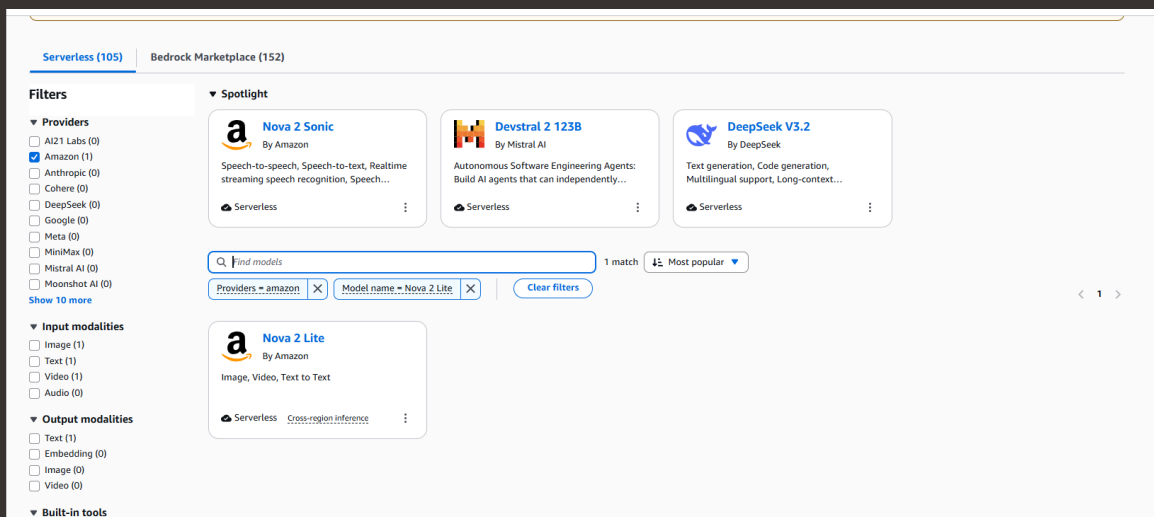
The key tools I used include AWS Bedrock agent which uses code interpreter, Python, Nova Lite foundation model, and a CSV file. The key concepts I learnt include prompt engineering, foundation model integration, API-based AI application development, conversation context management, inference parameter tuning, and building cloud-based generative AI solutions.

Challenges and wins

This project took me approximately sixty minutes. The most interesting part was learning how to use clear and concise instructions. It was most rewarding to learn how cloud, AI and analysis can be used in a single project.

Exploring Amazon Bedrock and Foundation Models

In this step, I am navigating to my AWS account, setting my region, navigating to Amazon Bedrock, finding Amazon Nova 2 lite and previewing it's features.



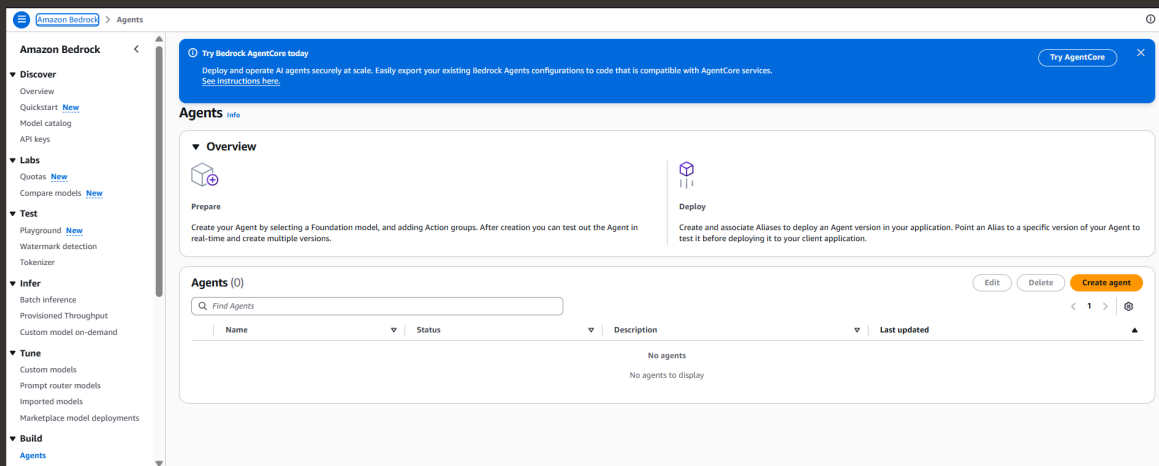
Understanding foundation models

A foundation model is a large AI model pre-trained on massive amounts of data. Instead of trying to train my own model from scratch, I can access one through an API. We are using Nova 2 Lite because it is Amazon's lightweight, fast, and cost-effective model, since I am on free tier account, I will be using it.



Discovering Bedrock Agents

A Bedrock Agent is an AI that can reason through multi-step tasks autonomously. Unlike a chatbot, it can decide what actions to take, execute code, call APIs and iterate on results.





Creating the AI Finance Agent

In this step, I will be creating an Amazon Bedrock Agent which will serve as a personal finance advisor, which I will also write the agent's natural language instructions and enable a code interpreter so the agent can write and execute python code.

The screenshot shows the 'Agent builder' interface with the following details:

- Agent details**
 - Agent name:** nextwork-finance-advisor (Valid characters are a-z, A-Z, 0-9, _ (underscore) and - (hyphen). The name can have up to 100 characters.)
 - Agent description - optional:** A personal finance AI that analyzes spending data (The description can have up to 200 characters.)
 - Agent resource role:** ☒ Create and use a new service role, ☐ Use an existing service role. The dropdown shows 'AmazonBedrockExecutionRoleForAgents_7T0JMWEM00U'.
 - Select model:** Amazon Nova Lite 1.0 On-demand.
 - Instructions for the Agent:** You are a personal finance advisor. Analyze spending data, categorize expenses, and suggest budgets. (This instruction must have a minimum of 40 characters.)
 - Additional settings:** (indicated by a right-pointing triangle icon)

Crafting the agent instructions

I wrote instructions that tell my agent to know that it is a personal finance advisor for analyzing spending data, categorize expenses and suggest budgets. This is important because the more clear and specific the instructions are, the better the agents performs.



nextwork-finance-advisor

Create AliasTestEdit in Agent Builder

Agent overview

Name	nextwork-finance-advisor	ID	WXI9ORJWLK
Description	A personal finance AI that analyzes spending data	Status	PREPARED
Creation date	May 29, 2026, 23:53 (UTC+03:00)	Last prepared	May 30, 2026, 00:19 (UTC+03:00)
Permissions	arn:aws:iam::176971016201:role/service-role/AmazonBedrockExecutionRoleForAgents_7T0JMWEMOOU		
User Input	DISABLED	Memory	Disabled
Idle session timeout	600 seconds	KMS key	-

Tags (0)

Manage Tags

Key	Value
No tags	

Manage tags

Versions (0)

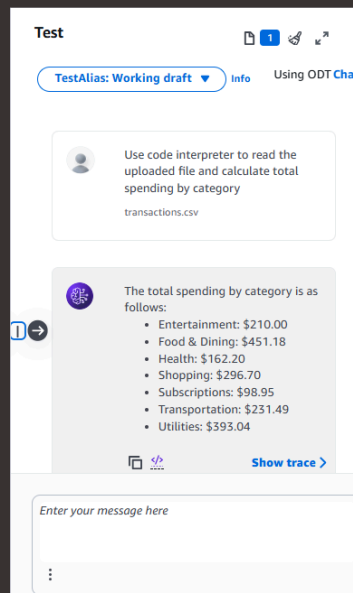
Info

Delete



Analyzing Spending Data with Code Interpreter

In this step, I will analyze my finances, one of the main purpose of this project. I will upload a transactions CSV and get a spending summary from my agent. I will also try getting budget recommendations from my agent.



How Code Interpreter processed the data

I uploaded a CSV file to code interpreter and gave it the instruction "Use code interpreter to read the uploaded file and calculate total spending by category". The agent used Code Interpreter to calculate spending by category. The output shows the spending by category as expected.



Iterating on Agent Instructions with Traces

I changed the instructions to "as a Personal finance advisor analyze spending, categorize expenses, suggest budgets, display all dollar amounts with two decimal places, and show each category's percentage of total spending". The output improved by showing the percentage breakdowns of each category.

```
43 ▼ "rationale": {
44   "text": "The User's goal is to calculate total spending by category and show percentage breakdowns
        .\n(2) The code interpreter has successfully calculated the total spending by category and the
        percentage breakdown of each category.\n(3) The next step is to provide the User with the
        calculated total spending by category and the percentage breakdown.\n(4) All steps in the action
        plan are complete.\n(5) No further action is required.\n(6) I have everything I need to provide
        the final response.",
45   "traceId": "ec3bfe1d-5818-4fe1-93c7-5e89a9ebaae3-1"
46 },
47 ▼ "observation": [
48   {
49     "finalResponse": {
50       "metadata": {
51         "endTime": "2026-05-29T21:57:09.664Z",
52         "operationTotalTimeMs": 5511,
53         "startTime": "2026-05-29T21:57:04.153Z"
54       },
55       "text": "The total spending by category and the percentage breakdown is as follows:\n-
        Entertainment: $210.00 (11.4%)\n- Food & Dining: $451.18 (24.5%)\n- Health: $162.20 (8.8%)\n-
        Shopping: $296.70 (16.1%)\n- Subscriptions: $98.95 (5.4%)\n- Transportation: $231.49 (12.6%
        )\n- Utilities: $393.04 (21.3%)"
56     },
57     "traceId": "ec3bfe1d-5818-4fe1-93c7-5e89a9ebaae3-1"
```




Enabling Cross-Session Agent Memory

In this project extension, I am enabling agent memory with session summarization. Cross-session memory is important because it allows the agent to recall past experiences, observations, and decisions to inform future actions and maintain context over time.

Memory - New

Test Agent

Using ODT [Change](#)

Memory allows an agent to retain conversational contexts across multiple sessions and recall past actions and behaviors. [See supported models](#)

Enable session summarization

Enabling session summarization will generate a summary of each test session. [View session summaries](#)

☒ Enabled

☐ Disabled

Memory duration

Set the maximum duration (in days) for which session summaries can be included in the agent's prompt context. Session summaries older than this limit will not be included.

days

Specify a duration between 1 and 365 days.

Maximum number of recent sessions

Set the maximum number of recent session summaries to include in the agent's prompt context. Session summaries beyond this limit will not be included. Limiting the number of sessions helps maintain prompt relevance and manage processing costs.

sessions

Minimum number of sessions is 1.

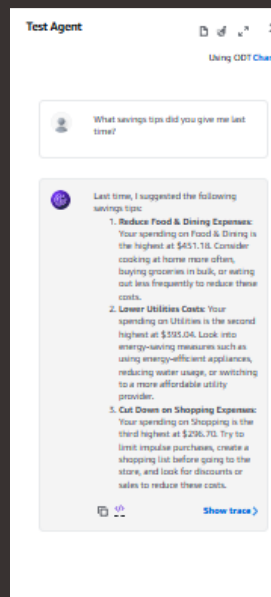
Enter your message here

⋮

Run

Session summarization vs full history

I configured the the memory section of the agent where i set the Storage duration to 30 days and set the Maximum recent sessions to 5. Summarization is useful because after every session, the agent creates a concise summary of key decisions and topics to use as references in the future sessions,



Testing cross-session recall

In this project extension, I tested the memory recall of my agent by asking it "Use code interpreter to analyze my spending and give me three specific ways I could save money", then started a new session. In the new session, I asked the agent "What savings tips did you give me last time?" The agent recalled the tips and outlined the tips exactly.



Wrapping Up

I did this project today to learn how to build and deploy conversational AI applications using Amazon Bedrock, integrate foundation models through APIs, and manage conversation context to create more natural and intelligent user interactions. Another skill I want to learn is how to enhance AI chatbots with external knowledge sources and tools, such as Retrieval-Augmented Generation (RAG), function calling, and agent-based workflows to handle more complex real-world tasks.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

